

UNIwersytet Rzeszowski
Wydział Nauk Ścisłych i Technicznych
Instytut Informatyki



Kacper Zoła

134993

Informatyka

Aplikacja do komunikacji tekstowej

Praca projektowa

Praca wykonana pod kierunkiem
mgr inż. Ewa Źesławska

Rzeszów 2026

Spis treści

1. Streszczenie	3
2. Opis założeń projektu	4
2.1. Cel oraz rozwiązany problem	4
2.2. Wymagania funkcjonalne projektu	4
3. Opis struktury projektu	5
3.1. Podział struktury projektu	5
3.2. Wykorzystane technologie	5
3.3. System kontroli wersji	6
3.4. Sterowniki użyte w projekcie.....	6
3.4.1. NuPackages:	6
3.5. Baza danych.....	7
3.5.1. Tabele	7
3.5.2. Funkcjonalność bazy danych.....	9
3.6. Funkcjonalność części serwerowej projektu	23
3.6.1. Program.cs.....	23
3.6.2. ConnectionManager.cs.....	24
3.6.3. TcpServer.cs.....	26
3.7. Funkcjonalność części użytkowej projektu.....	29
3.7.1. AuthWindow.....	29
3.7.2. ChatWindow.....	36
3.7.3. DataModels	50
3.7.4. Utilities	51
4. Harmonogram realizacji projektu	54
4.1. System kontroli wersji	54
4.2. Diagram Gantta	54
5. Prezentacja warstwy użytkowej	55
5.1. Interfejs graficzny.....	55
5.1.1. Okno logowania	55
5.1.2. Okno rejestracji	56
5.1.3. Okno komunikacji tekstowej.....	57
5.1.4. Okno ustawień.....	58
5.1.5. Okno dołączenia do pokoju	59
5.1.6. Okno stworzenia pokoju	60
5.2. Dane użytkownika dostępu do aplikacji/bazy danych.....	62
5.2.1. Baza danych - mongodb.com.....	62
5.2.2. Baza danych - maszyna wirtualna	62

5.2.3. Aplikacja	62
5.3. Instrukcja uruchomienia aplikacji	62
5.4. Co zostało zawarte w projekcie?	63
5.5. Możliwości rozwojowe.....	63
Bibliografia.....	64
Spis rysunków.....	65
Spis tabel.....	66
Spis listingów.....	67

1. Streszczenie

Streszczenie

Projekt polega na stworzeniu aplikacji pozwalającej na zdalną komunikację tekstową przez internet, projekt dzieli się na część serwerową i klienta. Klientą chcąc się zalogować, zarejestrować, wysłać wiadomość wysyła wiadomość walidującą do serwer, serwer sprawdza poprawność wysłanych danych i wykonuje odpowiednie operacje na bazie danych i wysyła wiadomość zwrotną.

Do przechowywania danych zastosowano nie relacyjną bazę danych MongoDB. Baza danych jest aktywana zdalnie na czas prezentacji projektu poprzez stronę <https://www.mongodb.com/>, dostępna jest także lokalnie w formie eksportu maszyny wirtualnej MongoDB debian.

Abstract

The project involves creating an application enabling remote text communication over the internet. The project is divided into server and client components. When the client wants to log in, register, or send a message, it sends a validation message to the server. The server validates the sent data, performs appropriate database operations, and sends a response message.

The non-relational MongoDB database is used to store the data. The database is activated remotely for the duration of the project presentation via the website <https://www.mongodb.com/>, also accessible locally as a virtual machine export MongoDB debian.

2. Opis założeń projektu

2.1. Cel oraz rozwiązany problem

Problem polega na braku możliwości komunikacji się z osobami na dalekie dystanse. Rozwiązaniem tego problemu jest stworzenie aplikacji pozwalającej na komunikację z osobami na daleki dystans poprzez użycie sieci internetowej.

2.2. Wymagania funkcjonalne projektu

1. Logowanie do systemu:

- Użytkownik ma możliwość zalogowania się do aplikacji.
- Użytkownik ma możliwość zarejestrowania się w bazie danych serwera.

2. Wiadomości tekstowe:

- Wyświetlanie wiadomości tekstowych.
- Wysyłanie wiadomości tekstowych.

3. Zarządzanie pokojami:

- Przechowywanie uczestników pokoju.
- Przechowywanie wysłanych wiadomości.
- Opuszczanie oraz usuwanie pokoi.

4. Obsługa bazy danych:

- Połączenie z bazą danych.
- Pobieranie danych z bazy danych.

5. Graficzny Interfejs Użytkownika:

- Graficzne okna aplikacji.

3. Opis struktury projektu

3.1. Podział struktury projektu

Projekt został podzielony na kilka części, które tworzą prostą, ale uporządkowaną strukturę.

1. Graficzny interfejs użytkownika

- Odpowiada ona za bezpośrednią interakcję z użytkownikiem, pomaga w wygodnym prezentowaniu danych, operacji poprzez obsługę działania np. okno logowania, okna komunikacji, wyświetlania dostępnych pokoi itp. Komunikaty, wywołane konkretnymi czynnościami, przyciski czy pola tekstowe są zbudowane przy pomocy WinUI.

2. Backend

- Ta warstwa odpowiada za funkcjonalność strony poprzez łączenie Frontendu (GUI) z funkcjonalnością wysyłania wiadomości do serwera. Przy wykonaniu odpowiednich akcji w GUI zostanie wywołana odpowiednia funkcja, zostanie wysłana wiadomość i odebrana odpowiedź od serwera.

3. MongoDB

- Operacje CRUD wykonywane są po stronie serwera poprzez C Sharp i platformę .NET.

4. Baza danych — na czas prezentacji projektu dostępna zdalnie.

3.2. Wykorzystane technologie

Do zrealizowania projektu użyto:

- C#
- .NET [3]
- WinUI 3 [4]
- BCrypt
- Microsoft EntityFrameworkCore
- MongoDB

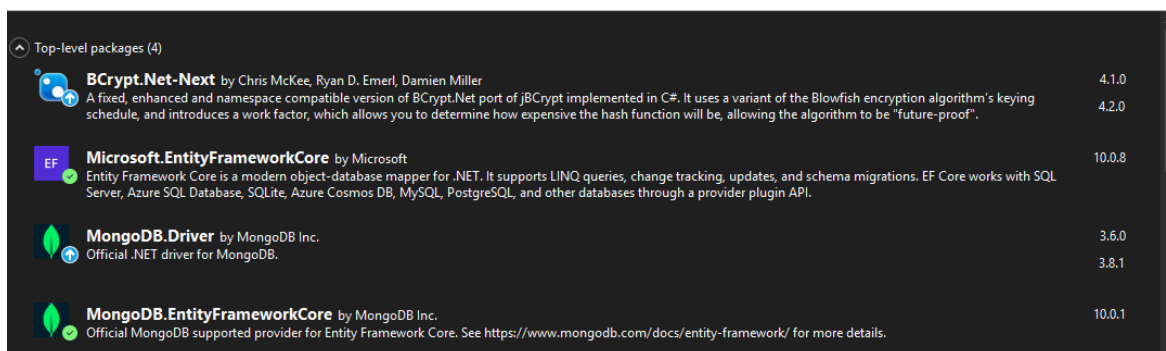
3.3. System kontroli wersji

Do kontrolowania wersji projektu użyto programu git, projekt przechowywany jest w formie zdalnej na platformie github.com pod adresem <https://github.com/K4-pi/Chat-app>

3.4. Sterowniki użyte w projekcie

3.4.1. NuPackages:

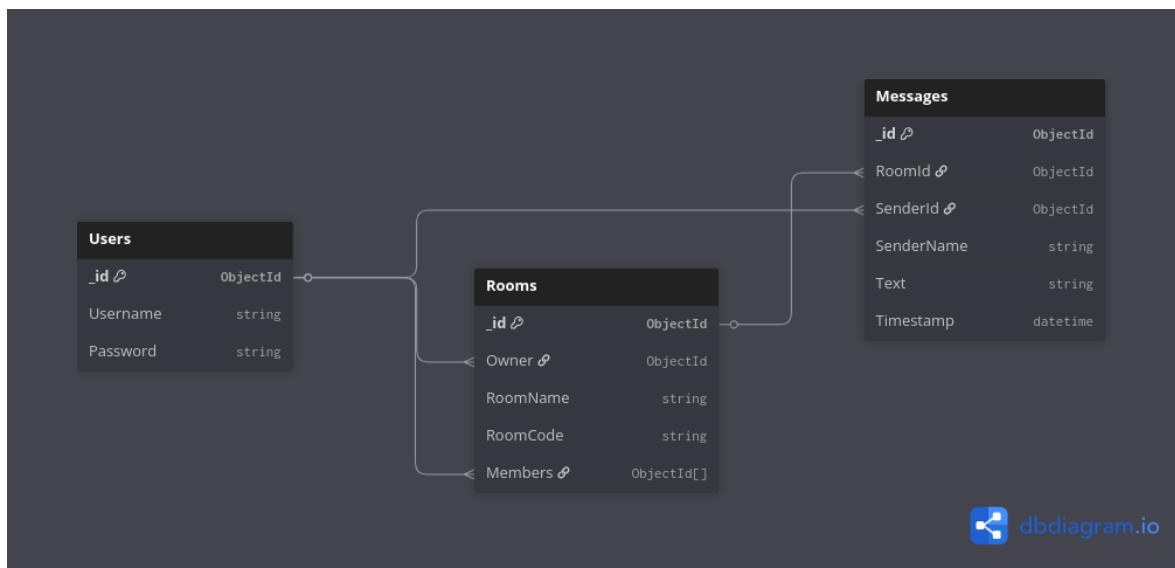
1. **BCrypt.Net-Next** [1]
2. **Microsoft.EntityFrameworkCore** [2]
3. **MongoDB.Driver** [5]
4. **MongoDB.EntityFrameworkCore**



Rysunek 3.1. NuPackages

3.5. Baza danych

Poniżej zamieszczony zrzut ekranu przedstawia relacje oraz strukturę bazy danych.



Rysunek 3.2. Zdjęcie przedstawiające strukturę bazy danych

3.5.1. Tabele

1. Tabela Users

Users – Przechowuje dane uwierzytelniające użytkowników. Hasła są składowane w formie zahashowanej przy użyciu algorytmu **BCrypt**.

- `_id`: Unikalny identyfikator obiektu (*ObjectId*).
- `Username`: Unikalna nazwa użytkownika (klucz wyszukiwania).
- `Password`: Hash hasła wraz z solą.

```

1 {
2   "_id": { "$oid": "69a9d8d1af00bbf6a6fd68f5" },
3   "Username": "User321",
4   "Password": "$2a$11$AtCT..."
5 }
  
```

Listing 3.1. Table users

2. Tabela Rooms

Rooms – Definiuje kanały komunikacyjne (pokoje).

- **Owner:** Referencja do `_id` w kolekcji *Users* (właściciel pokoju).
- **RoomName:** Publiczna nazwa pokoju.
- **RoomCode:** Kod dostępu (PIN) służący do autoryzacji dołączenia.
- **Members:** Tablica (*Array*) identyfikatorów *ObjectId*, reprezentująca listę uczestników.

```
1 {  
2   "_id": { "$oid": "69aef003820ada8df6bf10ae" },  
3   "Owner": { "$oid": "69a9d8d1af00bbf6a6fd68f5" },  
4   "RoomName": "General",  
5   "RoomCode": "general",  
6   "Members": [  
7     { "$oid": "69a9d8d1af00bbf6a6fd68f5" },  
8     { "$oid": "69aef6c0740748c077317d69" }  
9   ]  
10 }
```

Listing 3.2. Table rooms

3. Tabela Messages

Messages – Kolekcja przechowująca historię konwersacji.

- **RoomId:** Klucz obcy wiążący wiadomość z konkretnym pokojem.
- **SenderId:** Referencja do autora wiadomości.
- **SenderName:** Denormalizowane pole nazwy nadawcy (zapobiega kosztownym operacjom *\$lookup* podczas renderowania listy wiadomości).
- **Text:** Treść wiadomości.
- **Timestamp:** Znacznik czasu zapisu w formacie UTC.

```
1 {  
2   "_id": { "$oid": "69a37ae1432e01c78b2a22e2" },  
3   "RoomId": { "$oid": "69aef003820ada8df6bf10ae" },  
4   "SenderId": { "$oid": "69a9d8d1af00bbf6a6fd68f5" },  
5   "SenderName": "User321",  
6   "Text": "Hello World!",  
7   "Timestamp": { "$date": "2026-03-24T12:30:00.000Z" }  
8 }
```

Listing 3.3. Table messages

3.5.2. Funkcjonalność bazy danych

3.5.2.1. Klasa Database - konstruktor

Klasa zawiera funkcję globalną `__db` która służy do sprawnej komunikacji z bazą danych. Konstruktor klasy definiuje zmienną `connectionUri`, zawiera adres z nazwą użytkownika oraz hasłem do strony <https://www.mongodb.com/> na której uruchomiona jest baza danych, oraz wykonuje operacje połączenia z bazą danych.

```
1 using MongoDB.Bson;
2 using MongoDB.Driver;
3 using System.Diagnostics;
4 using System.Net.Sockets;
5 using System.Text;
6 using Microsoft.EntityFrameworkCore;
7
8 namespace ChatServer
9 {
10     internal class Database
11     {
12         private readonly ChatDbContext _db;
13
14         public Database()
15         {
16             const string connectionUri = "mongodb+srv://chat_db_user:
17             VbAcD6aY3FPrywXd@mainclusters.mpve4di.mongodb.net";
18             var client = new MongoClient(connectionUri);
19             var mongoDb = client.GetDatabase("ChatApp");
20             _db = ChatDbContext.Create(mongoDb);
21         }
22     }
23
24     ...
25 }
```

Listing 3.4. Database konstruktor

3.5.2.2. ChatDbContext

Klasa *ChatDbContext* zawiera kontekst bazy danych aplikacji czatu, dziedziczący po *DbContext* z Entity Framework Core. Skonfigurowany do pracy z MongoDB za pomocą *MongoDB.EntityFrameworkCore*. Pozwala na sprawne operowanie na bazie danych.

1. **Create(IMongoDatabase database)** - Tworzy i zwraca nową instancję *ChatDbContext* z istniejącego połączenia z bazą danych MongoDB. Eliminuje potrzebę ręcznego konfigurowania *DbContextOptionsBuilder* przy każdym tworzeniu kontekstu,
2. **ChatDbContext(DbContextOptions<ChatDbContext> options)** - konstruktor przekazuje opcje konfiguracyjne do klasy bazowej *DbContext*,
3. **OnModelCreating(ModelBuilder modelBuilder)** - określa, która klasa odpowiada której kolekcji w bazie danych. Np. klasa *User* → kolekcja *users*, klasa *Room* → kolekcja *rooms*, itd.

```

1 using Microsoft.EntityFrameworkCore;
2 using MongoDB.Driver;
3 using MongoDB.EntityFrameworkCore.Extensions;
4
5 namespace ChatServer
6 {
7     public class ChatDbContext : DbContext
8     {
9         public DbSet<User> Users { get; set; } = null!;
10        public DbSet<Room> Rooms { get; set; } = null!;
11        public DbSet<Message> Messages { get; set; } = null!;
12
13        public static ChatDbContext Create(IMongoDatabase database) =>
14            new(new DbContextOptionsBuilder<ChatDbContext>()
15                .UseMongoDB(database.Client, database.DatabaseNamespace.
16                    DatabaseName)
17                .Options);
18
19        public ChatDbContext(DbContextOptions<ChatDbContext> options) : base(
20            options) { }
21
22        protected override void OnModelCreating(ModelBuilder modelBuilder)
23        {
24            base.OnModelCreating(modelBuilder);
25            modelBuilder.Entity<User>().ToCollection("users");
26            modelBuilder.Entity<Room>().ToCollection("rooms");
27            modelBuilder.Entity<Message>().ToCollection("messages");
28        }
29    }
30 }

```

Listing 3.5. ChatDbContext

3.5.2.3. Models

Plik *Models* zawiera 3 klasy pomocnicze do operacji bazodanowych.

```
1 using MongoDB.Bson;
2 using MongoDB.Bson.Serialization.Attributes;
3
4 namespace ChatServer
5 {
6     public class User
7     {
8         [BsonId]
9         public ObjectId Id { get; set; }
10        public string Username { get; set; } = null!;
11        public string Password { get; set; } = null!;
12    }
13
14    public class Room
15    {
16        [BsonId]
17        public ObjectId Id { get; set; }
18        public ObjectId Owner { get; set; }
19        public string RoomName { get; set; } = null!;
20        public string RoomCode { get; set; } = null!;
21        public List<ObjectId> Members { get; set; } = new();
22    }
23
24    public class Message
25    {
26        [BsonId]
27        public ObjectId Id { get; set; }
28        public ObjectId RoomId { get; set; }
29        public ObjectId SenderId { get; set; }
30        public string SenderName { get; set; } = null!;
31        public string Text { get; set; } = null!;
32        public DateTime Timestamp { get; set; }
33    }
34 }
```

Listing 3.6. Models

3.5.2.4. Funkcja SendMessageHistoryAsync

Funkcja ta służy do wysłania historii wiadomości do użytkownika w celu wyświetlenia ich w aplikacji. Pobiera 2 parametry, id pokoju oraz *stream* klienta. Wykonuje operacje na tabeli pokoi i wiadomości. Pobiera 50 wiadomości (limit może zostać zwiększony bądź usunięty), jeżeli jakieś wiadomości z podanym id pokoju istnieją to zostają one wysłane do użytkownika który wysłał prośbę o historię wiadomości.

```
1 public async Task SendMessageHistoryAsync(string roomId, TcpClient client)
2 {
3     var oid = new ObjectId(roomId);
4
5     var messages = await _db.Messages
6         .Where(m => m.RoomId == oid)
7         .OrderBy(m => m.Timestamp)
8         .Take(50)
9         .ToListAsync();
10
11     if (messages.Count == 0) return;
12
13     foreach (var msg in messages)
14     {
15         await ConnectionManager.SendAsync(
16             $"MSG:{roomId}@{msg.SenderName}@{msg.Text}@{msg.Timestamp.
17             ToUniversalTime():HH:mm dd/MM/yyyy}",
18             client);
19     }
```

Listing 3.7. Funkcja SendMessageHistoryAsync

3.5.2.5. Funkcja GetUsersListAsync

Funkcja ta służy do pobrania listy użytkowników w pokoju o wysłanym id. Funkcja ta pobiera 1 parametr, id pokoju. Sprawdza czy podany pokój istnieje, jeżeli nie to zwraca odpowiedni komunikat. Jeśli w obecnym pokoju są użytkownicy to funkcja zwraca ciąg znaków z początkiem informującym o operacji oraz nazwami użytkownika oddzielonymi znakiem '@'.

```
1 public async Task<string> GetUsersListAsync(string roomId)
2 {
3     Console.WriteLine($"Room ID:{roomId}");
4
5     var oid = new ObjectId(roomId);
6     var room = await _db.Rooms.FirstOrDefaultAsync(r => r.Id == oid);
7
8     if (room == null) return "USERS_LIST:NONE";
9
10    var users = await _db.Users
11        .Where(u => room.Members.Contains(u.Id))
12        .ToListAsync();
13
14    var names = users.Select(u => u.Username);
15    return "USERS_LIST:" + string.Join("@", names);
16 }
```

Listing 3.8. Funkcja GetUsersListAsync

3.5.2.6. Funkcja GetUserRoomsAsync

Funkcja ta przyjmuje parametr id użytkownika i służy do odczytania z bazy danych pokoi do których przypisany jest użytkownik. Funkcja zwraca ciąg znaków z początkiem informującym o operacji oraz pokojami w formacie 'id pokoju, nazwa pokoju' oddzielonymi znakiem '@'.

```
1 public async Task<string> GetUserRoomsAsync(string userId)
2 {
3     var oid = new ObjectId(userId);
4
5     var rooms = await _db.Rooms
6         .Where(r => r.Members.Contains(oid))
7         .ToListAsync();
8
9     if (rooms.Count == 0) return "ROOM_LIST:NONE";
10
11     var roomStrings = rooms.Select(r => $"{r.Id},{r.RoomName},{r.RoomCode}");
12     return "ROOM_LIST:" + string.Join("@", roomStrings);
13 }
```

Listing 3.9. Funkcja GetUserRoomsAsync

3.5.2.7. Funkcja BroadcastToRoomAsync

Funkcja ta służy do wysłania nowo otrzymanych wiadomości do wszystkich aktualnie połączonych użytkowników, funkcja przyjmuje 4 parametry, id pokoju, treść wiadomości, czas wysłania wiadomości oraz id wysyłającego użytkownika. Sprawdza którzy aktywni użytkownicy są przypisani do pokoju i wysyła im wiadomość.

```
1 public async Task BroadcastToRoomAsync(string roomId, string messageText,
2     string sendTime, string senderId)
3 {
4     Console.WriteLine("Broadcast function start...");
5
6     var rid = new ObjectId(roomId);
7     var uid = new ObjectId(senderId);
8
9     var room = await _db.Rooms.FirstOrDefaultAsync(r => r.Id == rid);
10    if (room == null) return;
11
12    var user = await _db.Users.FirstOrDefaultAsync(u => u.Id == uid);
13    string senderName = user?.Username ?? "Unknown";
14
15    string protocolMessage = $"MSG:{roomId}@{senderName}@{messageText}@{
16        DateTime.Parse(sendTime):HH:mm dd/MM/yyyy}";
17    byte[] data = Encoding.UTF8.GetBytes(protocolMessage);
18
19    foreach (var memberId in room.Members)
20    {
21        Console.WriteLine($"Member ID in broadcast: {memberId}");
22        TcpClient? targetClient = ConnectionManager.GetClient(memberId.ToString());
23
24        if (targetClient != null && targetClient.Connected)
25        {
26            try
27            {
28                await targetClient.GetStream().WriteAsync(data);
29                Console.WriteLine($"Sent to member: {memberId}");
30            }
31            catch
32            {
33                Debug.WriteLine("Disconnected...");
34            }
35        }
36        else
37        {
38            Console.WriteLine("This client is not connected");
39        }
40    }
41 }
```

39 }

Listing 3.10. Funkcja BroadcastToRoomAsync

3.5.2.8. Funkcja StoreMessageAsync

Funkcja ta służy do zapisywania otrzymanych wiadomości, przyjmuje 4 parametry id pokoju do którego wiadomość została wysłana, treść wiadomości, czas wysłania, oraz *stream* klienta który wysłał tą wiadomość. Funkcja pobiera id użytkownika ze słownika (jeśli istnieje) który przechowuje *stream* wszystkich aktywnych użytkowników i przypisane do nich id. Sprawdza czy pokój o podanym id istnieje, jeśli tak wiadomość zostaje zapisa w bazie danych.

```
1 public async Task StoreMessageAsync(string roomId, string message, string
   sendTime, TcpClient client)
2 {
3     string senderId = ConnectionManager.GetUserId(client);
4     if (string.IsNullOrEmpty(senderId))
5     {
6         Debug.WriteLine("NULL sender");
7         return;
8     }
9
10    var rid = new ObjectId(roomId);
11    var uid = new ObjectId(senderId);
12
13    var room = await _db.Rooms.FirstOrDefaultAsync(r => r.Id == rid);
14    if (room == null)
15    {
16        Console.WriteLine($"Error: Room with id '{roomId}' not found");
17        return;
18    }
19
20    var user = await _db.Users.FirstOrDefaultAsync(u => u.Id == uid);
21    string senderName = user?.Username ?? "Unknown";
22
23    Console.WriteLine($"RoomID: {roomId}");
24    Console.WriteLine($"Sender ID: {senderId}");
25    Console.WriteLine($"Message: {message}");
26
27    _db.Messages.Add(new Message
28    {
29        RoomId = rid,
30        SenderId = uid,
31        SenderName = senderName,
32        Text = message,
33        Timestamp = DateTime.Parse(sendTime)
34    });
35
36    await _db.SaveChangesAsync();
37 }
```

Listing 3.11. Funkcja StoreMessageAsync

3.5.2.9. Funkcja DeleteRoomAsync

Funkcja ta służy do usuwania oraz opuszczania pokoju, przyjmuje 2 parametry nazwę pokoju oraz *stream* klienta wysyłającego polecenie. Funkcja bierze id użytkownika ze słownika, sprawdza czy pokój istnieje oraz czy użytkownik wysyłający polecenie jest właścicielem pokoju, jeżeli tak to pokój zostaje usunięty, jeżeli nie to sam użytkownik jest z niego usunięty. Funkcja zwraca odpowiedni komunikat w zależności do wykonanej akcji.

```
1 public async Task<string> DeleteRoomAsync(string roomName, TcpClient client)
2 {
3     Console.WriteLine($"Room:{roomName}");
4     string userId = ConnectionManager.GetUserId(client);
5     if (userId == null) return "DELETE_ROOM:FAILED";
6
7     var room = await _db.Rooms.FirstOrDefaultAsync(r => r.RoomName == roomName)
8         ;
9     if (room == null) return "DELETE_ROOM:FAILED";
10
11     var uid = new ObjectId(userId);
12
13     if (room.Owner == uid)
14     {
15         _db.Rooms.Remove(room);
16         await _db.SaveChangesAsync();
17         return "DELETE_ROOM:DELETED";
18     }
19     else
20     {
21         room.Members.Remove(uid);
22         await _db.SaveChangesAsync();
23         return "DELETE_ROOM:LEFT";
24     }
25 }
```

Listing 3.12. Funkcja DeleteRoomAsync

3.5.2.10. Funkcja JoinRoomAsync

Funkcja ta służy do przypisania użytkownika do pokoju w bazie danych. Przyjmuje 3 parametry nazwę pokoju, kod dostępu do pokoju, oraz *stream* klienta. Id użytkownika pobierane jest ze słownika aktywnych użytkowników. Funkcja sprawdza czy użytkownik ma poprawne id oraz czy kod dostępu do pokoju jest poprawny, jeżeli tak zostaje on przypisany do pokoju. Funkcja zwraca komunikat odpowiedni do wykonanej operacji.

```
1 public async Task<string> JoinRoomAsync(string roomName, string roomCode,
2     TcpClient client)
3 {
4     string userID = ConnectionManager.GetUserId(client);
5     if (userID == null) return "JOIN_ROOM:FAILED";
6
7     var room = await _db.Rooms.FirstOrDefaultAsync(r => r.RoomName == roomName)
8         ;
9     if (room == null) return "JOIN_ROOM:NO_ROOM";
10
11     var uid = new ObjectId(userID);
12     if (room.Members.Contains(uid)) return "JOIN_ROOM:FAILED"; // already a
13         member
14
15     room.Members.Add(uid);
16     await _db.SaveChangesAsync();
17     return "JOIN_ROOM:SUCCESS";
18 }
```

Listing 3.13. Funkcja JoinRoomAsync

3.5.2.11. Funkcja CreateRoomAsync

Funkcja ta służy do stworzenia pokoju. Funkcja ta przyjmuje 3 parametry nazwę pokoju, kod dostępu do pokoju, *stream* klienta wysyłającego wiadomość. Id użytkownika pobierane jest ze słownika aktywnych użytkowników, funkcja sprawdza czy pokój o danej nazwie już istnieje (jeżeli tak to zwraca błąd), pokój jest tworzony z id właściciela użytkownika który wysłał polecenie, o podanej nazwie i kodzie podanym przez użytkownika. Funkcja zwraca komunikat odpowiedni do wykonanej akcji.

```
1 public async Task<string> CreateRoomAsync(string roomName, string roomCode,
    TcpClient client)
2 {
3     string userID = ConnectionManager.GetUserId(client);
4     if (userID == null) return "CREATE_ROOM:FAILED";
5
6     var existing = await _db.Rooms.FirstOrDefaultAsync(r => r.RoomName ==
        roomName);
7     if (existing != null) return "CREATE_ROOM:EXISTS";
8
9     var uid = new ObjectId(userID);
10    _db.Rooms.Add(new Room
11    {
12        Owner = uid,
13        RoomName = roomName,
14        RoomCode = roomCode,
15        Members = new List<ObjectId> { uid }
16    });
17
18    await _db.SaveChangesAsync();
19    return "CREATE_ROOM:SUCCESS";
20 }
```

Listing 3.14. Funkcja CreateRoomAsync

3.5.2.12. Funkcja RegisterUser

Funkcja ta służy do rejestrowania nowych użytkowników. Przyjmuje 2 parametry, wysłane dane oraz *stream* klienta który wysłał polecenie. Funkcja dzieli dane na dwie części w miejscu w którym znajduje się znak '@', podzielone dane to hasło oraz nazwa użytkownika. Sprawdza czy użytkownik o podanej nazwie już istnieje (jeżeli tak to zwraca komunikat o niepowodzeniu), Funkcja zapisuje użytkownika o podanych informacjach w bazie danych z zahaszowanym hasłem. Dodany użytkownik od razu jest dodawany do pokoju 'general' który służy jako domyślny pokój dla każdego użytkownika. Funkcja zwraca komunikat odpowiedni do wykonanej operacji.

```
1 public async Task<string> RegisterUser(string data, TcpClient client)
2 {
3     string[] credentials = data.Split('@', 2);
4     string username = credentials[0];
5     string password = credentials[1];
6
7     var existing = await _db.Users.FirstOrDefaultAsync(u => u.Username ==
8         username);
9     if (existing != null) return "REGISTER_FAIL";
10
11     var userId = ObjectId.GenerateNewId();
12     _db.Users.Add(new User
13     {
14         Id = userId,
15         Username = username,
16         Password = BCrypt.Net.BCrypt.HashPassword(password)
17     });
18     await _db.SaveChangesAsync();
19
20     var general = await _db.Rooms.FirstOrDefaultAsync(r => r.RoomName == "
21         general");
22     if (general != null && !general.Members.Contains(userId))
23     {
24         general.Members.Add(userId);
25         await _db.SaveChangesAsync();
26         Console.WriteLine("User successfully added to room");
27     }
28     else
29     {
30         Console.WriteLine("User was already in the room");
31     }
32
33     return "REGISTER_SUCCESS";
34 }
```

Listing 3.15. Funkcja RegisterUser

3.5.2.13. Funkcja Authenticate

Funkcja ta służy do uwierzytelnienia użytkownika, przyjmuje 2 parametry, dane wysłane przez użytkownika oraz *stream* użytkownika wysyłającego wiadomość. Funkcja dzieli dane na dwie zmienne w miejscu w którym znajduje się znak '@', podzielone dane to nazwa użytkownika oraz hasło. Funkcja sprawdza czy użytkownik znajduje się w bazie danych oraz czy hasło zgadza się z przechowywanym hasłem. Jeżeli uwierzytelnianie przebiegło pomyślnie użytkownik jest dodawany do słownika aktywnych użytkowników i zwracany jest komunikat o poprawnym logowaniu, inaczej zwracany jest komunikat o nie udanym logowaniu.

```
1 public string Authenticate(string data, TcpClient client)
2 {
3     string[] credentials = data.Split('@', 2);
4     string username = credentials[0];
5     string password = credentials[1];
6
7     Console.WriteLine($"Username: {username}");
8     Console.WriteLine($"Password: {password}");
9
10    var user = _db.Users.FirstOrDefault(u => u.Username == username);
11
12    if (user != null && BCrypt.Net.BCrypt.Verify(password, user.Password))
13    {
14        string userId = user.Id.ToString();
15        ConnectionManager.AddUser(userId, client);
16        return $"AUTH_SUCCESS:{userId}";
17    }
18
19    return "AUTH_FAIL";
20 }
```

Listing 3.16. Funkcja Authenticate

3.6. Funkcjonalność części serwerowej projektu

3.6.1. Program.cs

Program.cs zawiera ustawienia adresu i portu serwera, oraz główną pętlę w której działa aplikacja. Jest to plik w którym serwer rozpoczyna prace.

```
1 using ChatServer;
2 using System.Net;
3 using System.Net.Sockets;
4
5 IPAddress IPAddress = IPAddress.Any;
6 int Port = 9000;
7
8 var listener = new TcpListener(IPAddress, Port);
9 listener.Start();
10 Console.WriteLine($"Server listening on {IPAddress}:{Port}");
11
12 while (true) //MAIN LOOP
13 {
14     var client = await listener.AcceptTcpClientAsync();
15     _ = TcpServer.ClientHandler(client);
16 }
```

Listing 3.17. Program.cs

3.6.2. ConnectionManager.cs

ConnectionManager jest klasą zawierającą funkcje związane z aktualizacją połączonych użytkowników, dane przechowywane są w formie *słownika* o kluczy typu *string* (nazwa użytkownika) oraz wartości typu *TcpClient* (*stream*, połączenie między komputerami). Klasa zawiera funkcje do dodawania, usuwania oraz pobierania danego użytkownika, jak i wysyłania wiadomości do ścieżki (*stream*).

```
1 using System.Net.Sockets;
2 using System.Text;
3
4 namespace ChatServer
5 {
6     public static class ConnectionManager
7     {
8         private static Dictionary<string, TcpClient> _onlineUsers = new
Dictionary<string, TcpClient>();
9
10        public static Dictionary<string, TcpClient> GetOnlineUsers()
11        {
12            return _onlineUsers;
13        }
14
15        public static void AddUser(string userId, TcpClient client)
16        {
17            _onlineUsers[userId] = client;
18        }
19
20        public static void RemoveUser(TcpClient client)
21        {
22            string uid = GetUserId(client);
23            _onlineUsers.Remove(uid);
24            Console.WriteLine($"Removed user with ID: {uid} from table");
25        }
26
27        public static TcpClient GetClient(string userId)
28        {
29            _onlineUsers.TryGetValue(userId, out var client);
30            return client!;
31        }
32
33        public static string GetUserId(TcpClient client)
34        {
35            var entry = _onlineUsers.FirstOrDefault(x => x.Value == client);
36            return entry.Key;
37        }
38        public static async Task SendAsync(string data, TcpClient client)
39        {
40            byte[] res = Encoding.UTF8.GetBytes(data + "\n");
41            await client.GetStream().WriteAsync(res, 0, res.Length);
```

```
42         }  
43     }  
44 }
```

Listing 3.18. ConnectionManager.cs

3.6.3. TcpServer.cs

TcpServer.cs jest jedną z najważniejszych klas ponieważ zawiera funkcjonalność do odbierania danych od użytkowników, tokenizowania tych wiadomości i wykonywaniu na nich odpowiednich operacji na bazie danych i użytkownikach.

```
1 using System.Net.Sockets;
2 using System.Text;
3
4 namespace ChatServer
5 {
6     public static class TcpServer
7     {
8         public static async Task ClientHandler(TcpClient client)
9         {
10             var stream = client.GetStream();
11             var buffer = new byte[4096];
12
13             Database database = new Database();
14
15             try
16             {
17                 while (true)
18                 {
19                     var ms = new MemoryStream();
20
21                     do
22                     {
23                         int bytesRead = await stream.ReadAsync(buffer);
24                         if (bytesRead == 0)
25                         {
26                             Console.WriteLine("Client disconnected");
27                             return;
28                         }
29                         ms.Write(buffer, 0, bytesRead);
30                     }
31                     while (stream.DataAvailable);
32
33                     string msg = Encoding.UTF8.GetString(ms.ToArray()).Trim();
34                     Console.WriteLine($"{msg}");
35
36                     string response;
37
38                     if (msg.StartsWith("MSG:"))
39                     {
40                         Console.WriteLine("Storing message...");
41                         msg = msg.Substring(4);
42
43                         //{roomID}@{text}@{time}"
```

```
44         string[] formatted = msg.Split('@', 3);
45         string roomID = formatted[0];
46         string msgText = formatted[1];
47         string sendTime = formatted[2];
48
49         await database.StoreMessageAsync(roomID, msgText, sendTime,
client);
50
51         Console.WriteLine($"Broadcasting message to room {roomID}");
52         string senderId = ConnectionManager.GetUserId(client);
53         _ = database.BroadcastToRoomAsync(roomID, msgText, sendTime,
senderId);
54     }
55     else if (msg.StartsWith("LOGIN:"))
56     {
57         Console.WriteLine("Authentication...");
58         msg = msg.Substring(6);
59
60         response = database.Authenticate(msg, client);
61         await ConnectionManager.SendAsync(response, client);
62     }
63     else if (msg.StartsWith("REGISTER:"))
64     {
65         Console.WriteLine("Registering user...");
66         msg = msg.Substring(9);
67
68         response = await database.RegisterUser(msg, client);
69         await ConnectionManager.SendAsync(response, client);
70     }
71     else if (msg.StartsWith("GET_ROOMS"))
72     {
73         Console.WriteLine("Sending list of rooms...");
74
75         string uid = ConnectionManager.GetUserId(client);
76         if (uid != null)
77         {
78             response = await database.GetUserRoomsAsync(uid);
79             await ConnectionManager.SendAsync(response, client);
80         }
81     }
82
83     .....
84
85 }
86 }
87 finally
88 {
89     client.Close();
90     ConnectionManager.RemoveUser(client);
```

```
91     }  
92 }  
93 }  
94 }
```

Listing 3.19. TcpServer.cs

3.7. Funkcjonalność części użytkowej projektu

3.7.1. AuthWindow

3.7.1.1. AuthWindow.xaml.cs

Jest to klasa która definiuje pierwsze okno przy włączeniu aplikacji, zdefiniowane tu są funkcjonalności okna, np. czy można zmieniać rozmiar okna.

```
1 using Microsoft.UI.Xaml;
2 using System;
3
4 // To learn more about WinUI, the WinUI project structure,
5 // and more about our project templates, see: http://aka.ms/winui-project-info.
6
7 namespace Chat
8 {
9     /// <summary>
10    /// An empty window that can be used on its own or navigated to within a
11    /// Frame.
12    /// </summary>
13    public sealed partial class AuthWindow : Window
14    {
15        private static Window? authWindow;
16
17        public AuthWindow()
18        {
19            authWindow = this;
20            IntPtr hWnd = WinRT.Interop.WindowNative.GetWindowHandle(this);
21
22            Microsoft.UI.WindowId windowId = Microsoft.UI.Win32Interop.
23            GetWindowIdFromWindow(hWnd);
24
25            Microsoft.UI.Windowing.AppWindow appWindow =
26            Microsoft.UI.Windowing.AppWindow.GetFromWindowId(
27            Microsoft.UI.Win32Interop.GetWindowIdFromWindow(hWnd)
28            );
29
30            appWindow.Resize(new Windows.Graphics.SizeInt32(450, 600));
31            var presenter = appWindow.Presenter as Microsoft.UI.Windowing.
32            OverlappedPresenter;
33            if (presenter != null)
34            {
35                presenter.IsResizable = false; // Disables dragging edges
36                presenter.IsMaximizable = false; // Disables the maximize
37                button
38            }
39        }
40    }
41}
```

```
36         InitializeComponent();
37         AuthFrame.Navigate(typeof(LoginPage));
38     }
39
40     public static void CloseRootWindow()
41     {
42         authWindow!.Close();
43     }
44 }
45 }
```

Listing 3.20. AuthWindow.cs

3.7.1.2. LoginPage.xaml.cs

Jest to klasa strony logowania do okna *Auth Winow* (można przełączyć się z niej na strone rejestracji), zawiera funkcjonalność przycisków oraz pól tekstowych. Użytkownik loguje się tutaj do aplikacji. Klasa pobiera dane podane przez użytkownika, waliduje je na podstawie wyznaczonych znaków i wysła zapytanie do serwera jeśli wszystkie dane są poprawne. Jeśli zapytanie zwróci wynik pozytywny użytkownik zostanie przeniesiony do okna komunikacji tekstowej, inczaje wyświetli użytkownikowi komunikat o nieudanym logowaniu.

```
1  using Microsoft.UI.Xaml;
2  using Microsoft.UI.Xaml.Controls;
3  using System;
4  using System.Diagnostics;
5
6  using Chat.Utilities;
7
8  namespace Chat
9  {
10     public sealed partial class LoginPage : Page
11     {
12         private string address = "127.0.0.1";
13         private int port = 9000;
14
15         public LoginPage()
16         {
17             InitializeComponent();
18         }
19
20         private async void SignUp_Click(object sender, RoutedEventArgs e)
21         {
22             this.Frame.Navigate(typeof(RegisterPage));
23         }
24
25         private async void LoginButton_Click(object sender, RoutedEventArgs e
26         {
```

```
27     string username = UsernameField.Text;
28     string password = PasswordField.Password;
29
30     if (username == "")
31     {
32         ErrorLoginText.Text = "No username provided!";
33     }
34     else if (password == "")
35     {
36         ErrorLoginText.Text = "No password provided!";
37     }
38     else
39     {
40         foreach (char c in "':/,.-_+=!$#?%^&*|(){}[]><")
41         {
42             if (username.Contains(c) || password.Contains(c))
43             {
44                 ErrorLoginText.Text = "No special symbols!";
45                 UsernameField.Text = "";
46                 PasswordField.Password = "";
47                 return;
48             }
49         }
50
51         LoginButton.IsEnabled = false;
52         var client = new TcpChatClient();
53         try
54         {
55             if (await client.ConnectAsync(address, port))
56             {
57                 string request = $"LOGIN:{username}@{password}";
58                 string response = await client.SendAndReceiveAsync(
request);
59
60                 if (response.StartsWith("AUTH_SUCCESS:"))
61                 {
62                     string userId = response.Substring(13);
63
64                     var chatWin = new MainWindow(userId, client);
65                     AuthWindow.CloseRootWindow();
66                     chatWin.Activate();
67                     return;
68                 }
69                 else
70                 {
71                     ErrorLoginText.Text = "Wrong username and/or
password";
72                     await client.CloseConnectionAsync();
73                 }

```

```
74         }
75         else
76         {
77             ErrorLoginText.Text = "Server connction error";
78             await client.CloseConnectionAsync();
79         }
80     }
81     catch(Exception ex)
82     {
83         await client.CloseConnectionAsync();
84         Debug.WriteLine($"Login exception: {ex}");
85     }
86     finally
87     {
88         LoginButton.IsEnabled = true;
89     }
90 }
91 }
92 }
93 }
```

Listing 3.21. LoginPage.cs

3.7.1.3. RegisterPage.xaml.cs

Jest to klasa strony rejestracji konta do okna *AuthWindow* (można przełączyć się z niej na stronę logowania), zawiera funkcjonalność przycisków oraz pól tekstowych. Użytkownik podaje dane dotyczące nazwy użytkownika oraz hasła. Dane te są walidowane na podstawie wyznaczonych znaków i zostaje wysłane zapytanie do serwera odnośnie zapisania nowego użytkownika do bazy danych. Jeżeli zapytanie zostaje zwrócone z wynikiem pozytywnym to oznacza to, że użytkownik został dodany do bazy danych i może się zalogować, w innym wypadku zostaje wyświetlony komunikat o nieudanie rejestracji.

```
1 using Chat.Utilities;
2 using Microsoft.UI.Xaml;
3 using Microsoft.UI.Xaml.Controls;
4 using Microsoft.UI.Xaml.Media;
5 using System;
6 using System.Diagnostics;
7
8 // To learn more about WinUI, the WinUI project structure,
9 // and more about our project templates, see: http://aka.ms/winui-project-info.
10
11 namespace Chat
12 {
13     /// <summary>
14     /// An empty page that can be used on its own or navigated to within a
15     /// Frame.
16     /// </summary>
17     public sealed partial class RegisterPage : Page
18     {
19         private string address = "127.0.0.1";
20         private int port = 9000;
21
22         public RegisterPage()
23         {
24             InitializeComponent();
25         }
26
27         private async void SignIn_Click(object sender, RoutedEventArgs e)
28         {
29             this.Frame.GoBack();
30         }
31
32         private async void RegisterButton_Click(object sender,
33             RoutedEventArgs e)
34         {
35             ErrorLoginText.Foreground = new SolidColorBrush(Colors.Red);
36
37             string username = UsernameField.Text;
```

```
36         string password = PasswordField.Password;
37
38         if (username == "")
39         {
40             ErrorLoginText.Text = "No username provided!";
41
42         }
43         else if (password == "")
44         {
45             ErrorLoginText.Text = "No password provided!";
46         }
47         else
48         {
49             foreach (char c in "':/,. -_+ $#!?%^&*|(){}[]><")
50             {
51                 if (username.Contains(c) || password.Contains(c))
52                 {
53                     ErrorLoginText.Text = "No special symbols!";
54                     UsernameField.Text = "";
55                     PasswordField.Password = "";
56                     RegisterButton.IsEnabled = true;
57                     return;
58                 }
59             }
60
61             RegisterButton.IsEnabled = false;
62             TcpChatClient client = new TcpChatClient();
63             try
64             {
65                 if (await client.ConnectAsync(address, port))
66                 {
67                     string request = $"REGISTER:{username}@{password}";
68                     string response = await client.SendAndReceiveAsync(
request);
69
70                     if (response.StartsWith("REGISTER_SUCCESS"))
71                     {
72                         UsernameField.Text = "";
73                         PasswordField.Password = "";
74
75                         ErrorLoginText.Foreground = new SolidColorBrush(
Microsoft.UI.Colors.Green);
76                         ErrorLoginText.Text = "Account created
successfully!";
77                     }
78                     else
79                     {
80                         ErrorLoginText.Text = "Username already taken!";
81                     }
82                 }
83             }
84             catch { }
```

```
82         }
83         else
84         {
85             ErrorLoginText.Text = "Server connction error";
86         }
87     }
88     catch(Exception ex)
89     {
90         Debug.WriteLine($"Register exception: {ex}");
91     }
92     finally
93     {
94         await client.CloseConnectionAsync();
95         RegisterButton.IsEnabled = true;
96     }
97 }
98 }
99 }
100 }
```

Listing 3.22. RegisterPage.cs

3.7.2. ChatWindow

Klasa *MainWindow* jest bardziej złożona gdyż zawiera w sobie główne funkcjonalności związane z komunikacją tekstową, zostanie dlatego opisana funkcjami.

3.7.2.1. ChatWindow.xaml.cs

Początek klasy z konstruktorem definiuje globalne zmienne potrzebne do pracy aplikacji oraz ustawienia tworzonego okna.

```
1 using Chat.DataModels;
2 using Chat.Utilities;
3 using Microsoft.UI.Xaml;
4 using Microsoft.UI.Xaml.Controls;
5 using Microsoft.UI.Xaml.Input;
6 using System;
7 using System.Collections.ObjectModel;
8 using System.Diagnostics;
9 using System.Threading.Tasks;
10
11 // To learn more about WinUI, the WinUI project structure,
12 // and more about our project templates, see: http://aka.ms/winui-project-info.
13
14 namespace Chat
15 {
16     /// <summary>
17     /// An empty window that can be used on its own or navigated to within a
18     /// Frame.
19     /// </summary>
20     public sealed partial class MainWindow : Window
21     {
22         private TcpChatClient client;
23         public ObservableCollection<ChatRoom> userRooms { get; set; } = new
24             ObservableCollection<ChatRoom>();
25         public ObservableCollection<Message> Messages { get; } = new
26             ObservableCollection<Message>();
27         public ObservableCollection<User> UsersInRoom { get; } = new
28             ObservableCollection<User>();
29
30         private string? currentRoomId; //general
31         private bool isShowed = false;
32
33         public MainWindow(string userId, TcpChatClient _client)
34         {
35             IntPtr hWnd = WinRT.Interop.WindowNative.GetWindowHandle(this);
36
37             Microsoft.UI.WindowId windowId = Microsoft.UI.Win32Interop.
38                 GetWindowIdFromWindow(hWnd);
```

```
35         Microsoft.UI.Windowing.AppWindow appWindow =
36             Microsoft.UI.Windowing.AppWindow.GetFromWindowId(
37                 Microsoft.UI.Win32Interop.GetWindowIdFromWindow(hWnd)
38             );
39
40         appWindow.Resize(new Windows.Graphics.SizeInt32(1200, 600));
41
42         InitializeComponent();
43         Title = $"UserID: {userId}";
44
45         client = _client;
46         _ = client.ListenAsync(OnMessageReceived);
47         _ = client.SendAsync("GET_ROOMS");
48
49         RoomList.ItemsSource = userRooms;
50     }
51     ...
```

Listing 3.23. ChatWindow-main.cs

3.7.2.2. OnMessageReceived

Jest to jedna z najważniejszych funkcji klasy, definiuje ona co aplikacja ma zrobić przy uzyskaniu wiadomości od serwera. Przez to jak działa optymalizacja ścieżek komunikacji, pewne wiadomości mogą zostać dostarczone sklejone z innymi wiadomościami, dlatego są one cięte na końcach i dla każdej osobnej wiadomości wykonywana jest funkcja sprawdzająca. Początek każdej wiadomości jest sprawdzany i na podstawie nazwy operacji która znajduje się na początku wiadomości, wykonywana jest odpowiednia operacja. Obsługiwane operacje:

1. **MSG** → wiadomości wysyłane przez użytkowników, wyświetlane na ekranie.
2. **ROOM_LIST** → lista dostępnych pokoi dla użytkownika, zapisywana w pamięci.
3. **USERS_LIST** → lista użytkowników w danym pokoju, wyświetlana w panelu bocznym.
4. **CRETE_ROOM** → wiadomość o tym czy udało się utworzyć nowy pokój
5. **JOIN_ROOM** → wiadomość o tym czy udało się dołączyć do pokoju
6. **DELETE_ROOM** → wiadomość o tym czy udało się usunąć pokój

```
1 public async void OnMessageReceived(string msg)
2 {
3     Debug.WriteLine($"OnMessageReceived: {msg}");
4
5     string[] splitedMessages = msg.Split('\n', StringSplitOptions.
        RemoveEmptyEntries);
6
7     foreach (string s in splitedMessages)
8     {
9         // "MSG:roomId@User@Hello World!@time"
10        if (s.StartsWith("MSG:"))
11        {
12            // "roomId@User@Hello World!@time"
13            string formatted = s.Substring(4); // Remove "MSG:" prefix
14
15            // "[roomId] [User] [Hello World!] [time]"
16            string[] content = formatted.Split('@', 4);
17
18            string roomId = content[0];
19            string user = content[1];
20            string text = content[2];
21            string time = content[3];
22
23            Debug.WriteLine($"roomId: {roomId}");
24            Debug.WriteLine($"current room id: {currentRoomId}");
25
26            if (roomId != currentRoomId) return;
27
28            this.DispatcherQueue.TryEnqueue(() =>
```

```
29     {
30         Messages.Add(new Message
31         {
32             Username = user,
33             Text = text,
34             SentAt = time
35         });
36
37         MessageList.UpdateLayout();
38         MessageScrollView.ChangeView(null, MessageScrollView.
ScrollableHeight, null);
39     });
40 }
41 else if (s.StartsWith("ROOM_LIST:"))
42 {
43     string content = s.Substring(10);
44     if (content == "NONE") return;
45
46     string[] rooms = content.Split('@');
47
48     this.DispatcherQueue.TryEnqueue(() =>
49     {
50         RoomList.ItemsSource = null; // Prevents refreshing every time new
room is added
51
52         userRooms.Clear();
53         Debug.WriteLine("Your rooms:");
54
55         foreach (var r in rooms)
56         {
57             Debug.WriteLine(r);
58
59             var tokens = r.Split(',', 3);
60             var newRoom = new ChatRoom
61             {
62                 Id = tokens[0],
63                 Name = tokens[1],
64                 Code = tokens[2]
65             };
66             userRooms.Add(newRoom);
67         }
68
69         RoomList.ItemsSource = userRooms;
70     });
71 }
72 else if (s.StartsWith("USERS_LIST:"))
73 {
74     string content = s.Substring(11);
75     if (content == "NONE") return;
```

```
76
77     string[] users = content.Split('@');
78
79     Debug.WriteLine("users list:");
80     foreach (var u in users)
81     {
82         Debug.WriteLine($"USER {u}");
83     }
84
85     this.DispatcherQueue.TryEnqueue(() =>
86     {
87         UsersInRoom.Clear();
88
89         foreach (var u in users)
90         {
91             UsersInRoom.Add(new User
92             {
93                 Username = u
94             });
95         }
96
97         UsersList.UpdateLayout();
98         UserScrollView.ChangeView(null, UserScrollView.ScrollableHeight,
99 null);
100     });
101     }
102     else if (s.StartsWith("CREATE_ROOM:"))
103     {
104         string content = s.Substring(12);
105         if (content == "SUCCESS")
106         {
107             await client.SendAsync("GET_ROOMS");
108             await ShowAlert("SUCCESS", "Created room");
109         }
110         else if (content == "EXISTS")
111         {
112             await ShowAlert("ERROR", "Room with that name already exists");
113         }
114         else
115         {
116             await ShowAlert("ERROR", "Failed to create room");
117         }
118     }
119     else if (s.StartsWith("JOIN_ROOM:"))
120     {
121         string content = s.Substring(10);
122         if (content == "SUCCESS")
123         {
124             await client.SendAsync("GET_ROOMS");
```

```
124         await ShowAlert("SUCCESS", "Joined room");
125     }
126     else if (content == "NO_ROOM")
127     {
128         await ShowAlert("ERROR", "Room doesn't exists");
129     }
130     else
131     {
132         await ShowAlert("ERROR", "Failed to join a room");
133     }
134 }
135 else if (s.StartsWith("DELETE_ROOM:"))
136 {
137     string content = s.Substring(12);
138     if (content == "DELETED")
139     {
140         await client.SendAsync("GET_ROOMS");
141         await ShowAlert("SUCCESS", "Deleted room");
142     }
143     else if (content == "LEFT")
144     {
145         await client.SendAsync("GET_ROOMS");
146         await ShowAlert("SUCCESS", "Left room");
147     }
148     else
149     {
150         await ShowAlert("ERROR", "Room delete/leave problem");
151     }
152 }
153 }
154 }
```

Listing 3.24. OnMessageReceived

3.7.2.3. RoomList_SelectionChanged

Funkcja ta wywoływana jest podczas gdy użytkownik kliknie na pokój wyświetlany w GUI, wysyła ona zapytanie do bazy danych o historie wiadomości w celu zaktualizowania ich, oraz zapytanie o użytkowników przypisanych do danego pokoju w celu aktualizacji listy użytkowników.

```
1 private async void RoomList_SelectionChanged(object sender,
2     SelectionChangedEventArgs e)
3 {
4     if (RoomList.SelectedItem is ChatRoom selectedRoom)
5     {
6         currentRoomId = selectedRoom.Id;
7         Messages.Clear();
8         UsersInRoom.Clear();
9
10        await client.SendAsync($"GET_HISTORY:{currentRoomId}"); // Update
11        messages
12        Debug.WriteLine($"Switched to room: {selectedRoom.Name} ({currentRoomId})
13        ");
14
15        await Task.Delay(100);
16        await client.SendAsync($"GET_USERS_LIST:{currentRoomId}"); // Update
17        users
18    }
19 }
```

Listing 3.25. RoomListSelectionChanged

3.7.2.4. LogoutButton_Click

Funkcja ta podpięta jest pod przycisk wylogowania, wysyła wiadomość do serwera o wylogowaniu użytkownika, zamyka obecne okno i otwiera okno logowania.

```
1 public async void LogoutButton_Click(object sender, RoutedEventArgs e)
2 {
3     await client.SendAsync("DISCONNECT");
4     var authWindow = new AuthWindow();
5     this.Close();
6     authWindow.Activate();
7 }
```

Listing 3.26. LogoutButtonClick

3.7.2.5. SendMessage

Funkcja odpowiadająca za wysłanie wiadomości z pola tekstowego do serwera, sprawdzane jest tu czy użytkownik jest w pokoju, jeżeli nie zostaje wyświetlony komunikat, w innym wypadku wiadomość zostaje wysłana do serwera z obecnym roomID oraz czasem utworzenia wiadomości. Pole tekstowe jest czyszczone.

```
1 private async Task SendMessage()
2 {
3     /*
4         Not adding message to our own list because
5         server will broadcast it back
6     */
7     if (currentRoomId == null)
8     {
9         await ShowAlert("ERROR", "You need to choose room before you send message");
10        return;
11    }
12
13    string text = MessageInput.Text.Trim();
14    if (string.IsNullOrEmpty(text)) return;
15
16    try
17    {
18        await client.SendAsync($"MSG:{currentRoomId}@{text}@{DateTime.UtcNow.ToString()}");
19    }
20    catch (Exception ex)
21    {
22        Debug.WriteLine($"SendMessageException:{ex.Message}");
23    }
24    MessageInput.Text = "";
25 }
```

Listing 3.27. SendMessage

3.7.2.6. SendButton_Click

Funkcja ta jest podłączona do przycisku wysłania wiadomości, przy kliknięciu przycisku wywołuje funkcję *SendMessage()*

```
1 public async void SendButton_Click(object sender, RoutedEventArgs e)
2 {
3     await SendMessage();
4 }
```

Listing 3.28. SendButtonClick

3.7.2.7. MessageInput_KeyDown

Funkcja ta wywołuje funkcję *SendMessage()* przy kliknięciu przycisku *Enter*

```
1 private void MessageInput_KeyDown(object sender, KeyRoutedEventArgs e)
2 {
3     if (e.Key == Windows.System.VirtualKey.Enter)
4     {
5         _ = SendMessage();
6         e.Handled = true;
7     }
8 }
```

Listing 3.29. MessageInputKeyDown

3.7.2.8. DeleteRoomButton_Click

Funkcja ta powoduje wysłanie żądania o usunięcie pokoju.

```
1 private async void DeleteRoomButton_Click(object sender, RoutedEventArgs e)
2 {
3     var button = sender as Button;
4
5     if (button?.DataContext is ChatRoom selectedRoom)
6     {
7         await client.SendAsync($"DELETE_ROOM:{selectedRoom.Name}");
8     }
9 }
```

Listing 3.30. DeleteRoom

3.7.2.9. JoinRoomButton_Click

Funkcja ta powoduje utworzenie dialogu przy kliknięciu przycisku dołączenia do pokoju. Utworzony dialog zawiera dwa pola tekstowe oraz dwa przyciski, użytkownik podaje nazwę pokoju oraz kod dostępu, przy kliknięciu przycisku "Join" aplikacja wysyła żądanie o dołączenie do pokoju o podanej nazwie i kodzie wysłanej w formie wiadomości ciągu znaków. Jeżeli nie wszystkie pola zostały uzupełnione zostanie wyświetlony komunikat.

```
1 private async void JoinRoomButton_Click(object sender, RoutedEventArgs e)
2 {
3     TextBox roomNameInput = new TextBox
4     {
5         Header = "Name",
6         //PlaceholderText = "insert room name",
7         Margin = new Thickness(0, 0, 0, 10)
8     };
9
10    TextBox roomCodeInput = new TextBox
11    {
12        Header = "Code"
13        //PlaceholderText = "insert room code"
14    };
15
16    StackPanel panel = new StackPanel();
17    panel.Children.Add(roomNameInput);
18    panel.Children.Add(roomCodeInput);
19
20    ContentDialog dialog = new ContentDialog
21    {
22        Title = "Join room",
23        Content = panel,
24        PrimaryButtonText = "Join",
25        CloseButtonText = "Cancel",
26        DefaultButton = ContentDialogButton.Primary,
27        XamlRoot = this.Content.XamlRoot
28    };
29
30    ContentDialogResult result = await dialog.ShowAsync();
31
32    if (result == ContentDialogResult.Primary)
33    {
34        string roomCode = roomCodeInput.Text;
35        string roomName = roomNameInput.Text;
36
37        if (!string.IsNullOrEmpty(roomCode) && !string.IsNullOrEmpty(roomName))
38        {
39            await client.SendAsync($"JOIN_ROOM:{roomName}@{roomCode}");
40        }
41        else
```

```
42     {
43         await showAlert("ERROR", "You need to provide name and code");
44     }
45 }
46 }
```

Listing 3.31. JoinRoom

3.7.2.10. CreateRoomButton_Click

Funkcja ta powoduje utworzenie dialogu przy kliknięciu przycisku utworzenia pokoju. Utworzony dialog zawiera dwa pola tekstowe oraz dwa przyciski, użytkownik podaje nazwę pokoju oraz kod dostępu, przy kliknięciu przycisku "Create" aplikacja wysyła żądanie o utworzenie pokoju o podanej nazwie i kodzie wysłanej w formie wiadomości ciągu znaków. Jeżeli nie wszystkie pola zostały uzupełnione zostanie wyświetlony komunikat.

```
1 private async void CreateRoomButton_Click(object sender, RoutedEventArgs e)
2 {
3     TextBox roomNameInput = new TextBox
4     {
5         Header = "Name",
6         //PlaceholderText = "insert room name",
7         Margin = new Thickness(0, 0, 0, 10)
8     };
9
10    TextBox roomCodeInput = new TextBox
11    {
12        Header = "Code"
13        //PlaceholderText = "insert room code"
14    };
15
16    StackPanel panel = new StackPanel();
17    panel.Children.Add(roomNameInput);
18    panel.Children.Add(roomCodeInput);
19
20    ContentDialog dialog = new ContentDialog
21    {
22        Title = "Create room",
23        Content = panel,
24        PrimaryButtonText = "Create",
25        CloseButtonText = "Cancel",
26        DefaultButton = ContentDialogButton.Primary,
27        XamlRoot = this.Content.XamlRoot
28    };
29
30    ContentDialogResult result = await dialog.ShowAsync();
31
32    if (result == ContentDialogResult.Primary)
33    {
```

```

34     string roomName = roomNameInput.Text;
35     string roomCode = roomCodeInput.Text;
36
37     if (!string.IsNullOrEmpty(roomCode) && !string.IsNullOrEmpty(roomName))
38     {
39         await client.SendAsync($"CREATE_ROOM:{roomName}@{roomCode}");
40     }
41     else
42     {
43         await ShowAlert("ERROR", "You need to provide name and code");
44     }
45 }
46 }

```

Listing 3.32. CreateRoom

3.7.2.11. SettingsButton_Click

Funkcja ta powoduje utworzenie dialogu (okna) które, zawiera trzy pola tekstowe oraz dwa przyciski. Okno to pozwala na zmianę nazwy użytkownika poprzez wpisanie nowej nazwy użytkownika w polu tekstowym oraz wciśnięciu przycisku *check*, zapytanie o zmianę nazwy użytkownika jest walidowane oraz wysłane do serwera. Zmiana hasła działa na takiej samej.

```

1     .....
2
3     ContentDialog dialog = new ContentDialog
4     {
5         Title = "Update Account",
6         Content = panel,
7         PrimaryButtonText = null,
8         CloseButtonText = "Close",
9         DefaultButton = ContentDialogButton.Primary,
10        XamlRoot = this.Content.XamlRoot
11    };
12
13    string regex = "!@#%$%^&*()_-=+/?.,<>;:[]{}\\|\'\"";
14
15    usernameButton.Click += async (s, args) =>
16    {
17        string usernameString = newUsername.Text;
18
19        if (usernameString.Length < 4)
20        {
21            newUsername.Description = "Username must be at least 4 characters
22";
23        }
24        else if (usernameString.Any(c => regex.Contains(c)))
25        {
26            newUsername.Description = "Special characters are not allowed";
27        }
28    }

```

```
27         else
28         {
29             _ = client.SendAsync($"CHANGE_USERNAME:{usernameString}");
30
31             newUsername.Description = "Username changed";
32             newUsername.Text = "";
33         }
34     };
35
36     passwordButton.Click += (s, args) =>
37     {
38         string newPasswordString = newPassword.Text;
39
40         if (newPasswordString.Length < 4)
41         {
42             newPassword.Description = "Passowrd must be at least 4 characters
43 ";
44         }
45         else if (newPasswordString.Any(c => regex.Contains(c)))
46         {
47             newPassword.Description = "Special characters are not allowed";
48         }
49         else
50         {
51             _ = client.SendAsync($"CHANGE_PASSWORD:{newPasswordString}");
52
53             newPassword.Description = "Password changed";
54             newPassword.Text = "";
55         }
56     };
57
58     ContentDialogResult result = await dialog.ShowAsync();
```

Listing 3.33. Settings

3.7.2.12. ShowAlert

Funkcja ta służy do tworzenia dialogów informujących użytkownika o problemach, które występują przy używaniu aplikacji.

```
1 private async Task ShowAlert(string title, string message)
2 {
3     if (isShown) return;
4
5     isShown = true;
6     ContentDialog errorDialog = new ContentDialog
7     {
8         Title = title,
9         Content = message,
10        CloseButtonText = "OK",
11        DefaultButton = ContentDialogButton.Close,
12        XamlRoot = this.Content.XamlRoot
13    };
14
15    await errorDialog.ShowAsync();
16    isShown = false;
17 }
```

Listing 3.34. ShowAlert

3.7.3. DataModels

Są to klasy, które służą jako szablon pod przechowywane dane związane z komunikacją tekstową, wiadomości, użytkownicy, pokoje.

```
1 namespace Chat.DataModels
2 {
3     public class Message
4     {
5         public string? Username { get; set; }
6         public string? Text { get; set; }
7         public string? SentAt { get; set; }
8     }
9 }
```

Listing 3.35. Message

```
1 namespace Chat.DataModels
2 {
3     public class ChatRoom
4     {
5         public string? Id { get; set; }
6         public string? Name { get; set; }
7         public string? Code { get; set; }
8     }
9 }
```

Listing 3.36. ChatRoom

```
1 namespace Chat.DataModels
2 {
3     public class User
4     {
5         public string? Username { get; set; }
6     }
7 }
```

Listing 3.37. User

3.7.4. Utilities

Klasa *TcpChatClient* definiuje ścieżkę komunikacji oraz klienta TCP do komunikacji z serwerem. Definiuje także funkcje służące do nasłuchiwania przychodzących wiadomości w tle, wysyłania wiadomości, oraz zamykania połączenia.

```
1 using System;
2 using System.Diagnostics;
3 using System.IO;
4 using System.Net.Sockets;
5 using System.Text;
6 using System.Threading.Tasks;
7
8 namespace Chat.Utilities
9 {
10     public class TcpChatClient
11     {
12         private TcpClient? client;
13         private NetworkStream? stream;
14
15         public async Task<bool> ConnectAsync(string ip, int port)
16         {
17             try
18             {
19                 client = new TcpClient();
20                 await client.ConnectAsync(ip, port);
21                 stream = client.GetStream();
22                 return true;
23             }
24             catch
25             {
26                 return false;
27             }
28         }
29
30         public async Task CloseConnectionAsync()
31         {
32             try
33             {
34                 stream?.Close();
35                 client?.Close();
36                 client = null;
37             }
38             catch (Exception ex)
39             {
40                 Debug.WriteLine($"Error during cleanup: {ex.Message}");
41             }
42         }
43     }
```

```
44 // Listen after login
45 public async Task ListenAsync(Action<string> onMessageReceived)
46 {
47     Debug.WriteLine("Started listen loop");
48     try
49     {
50         if (stream == null || client == null) return;
51
52         while (client.Connected)
53         {
54             var ms = new MemoryStream(); // Needs to be here to 'clear' it
55             byte[] buffer = new byte[2048];
56             int bytesRead = 0;
57
58             do
59             {
60                 bytesRead = await stream.ReadAsync(buffer, 0, buffer.Length);
61                 if (bytesRead == 0) return;
62                 ms.Write(buffer, 0, bytesRead);
63             }
64             while (stream.DataAvailable);
65
66             string receivedData = Encoding.UTF8.GetString(ms.ToArray()).Trim();
67             onMessageReceived?.Invoke(receivedData); // Starts 'action' with
received message
68         }
69     }
70     catch (Exception ex)
71     {
72         Debug.WriteLine($"Disconnected: {ex.Message}");
73     }
74     Debug.WriteLine("Ended listen loop");
75 }
76
77 public async Task SendAsync(string message)
78 {
79     if (stream == null) return; // "ERROR:Not Connected"
80
81     try
82     {
83         byte[] data = Encoding.UTF8.GetBytes(message + '\n');
84         await stream.WriteAsync(data, 0, data.Length);
85     }
86     catch (Exception ex)
87     {
88         Debug.WriteLine($"SendAsyncException:{ex.Message}");
89     }
90 }
91
```

```
92     public async Task<string> SendAndReceiveAsync(string message)
93     {
94         try
95         {
96             if (stream == null) return "ERROR:Not Connected";
97
98             // Send
99             byte[] data = Encoding.UTF8.GetBytes(message);
100             await stream.WriteAsync(data, 0, data.Length);
101
102             // Receive - use of MemoryStream() prevents data loss problem
103             var ms = new MemoryStream();
104             byte[] buffer = new byte[2048];
105
106             do
107             {
108                 int bytesRead = await stream.ReadAsync(buffer, 0, buffer.Length);
109                 if (bytesRead == 0) break;
110                 ms.Write(buffer, 0, bytesRead);
111             }
112             while (stream.DataAvailable);
113
114             return Encoding.UTF8.GetString(ms.ToArray()).Trim();
115         }
116         catch (Exception ex)
117         {
118             Debug.WriteLine($"SendAndReceiveAsyncException{ex.Message}");
119         }
120         return "ERROR:catch";
121     }
122 }
123 }
```

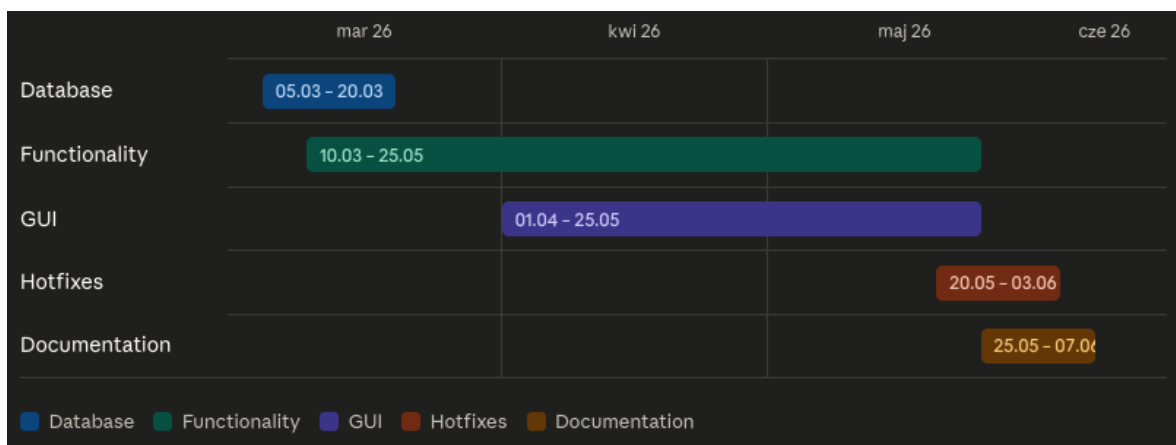
Listing 3.38. Tcp

4. Harmonogram realizacji projektu

4.1. System kontroli wersji

Do realizacji projektu użyto aplikacji Git jako system kontroli wersji, repozytorium projektu jest zapisane zdalnie na platformie Github pod adresem <https://github.com/K4-pi/Chat-app>. Diagram Gantta przedstawia etapy pracy nad projektem.

4.2. Diagram Gantta

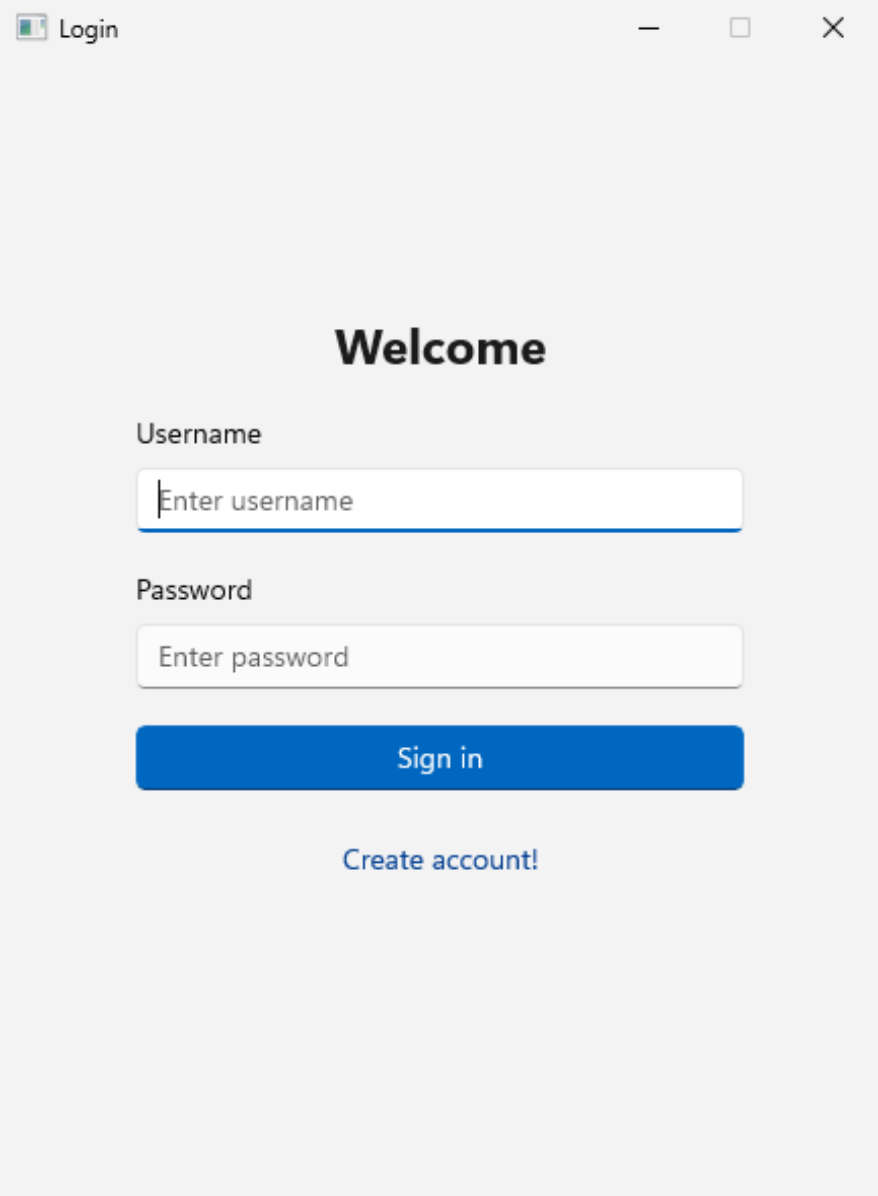


Rysunek 4.1. Diagram Gantta

5. Prezentacja warstwy użytkowej

5.1. Interfejs graficzny

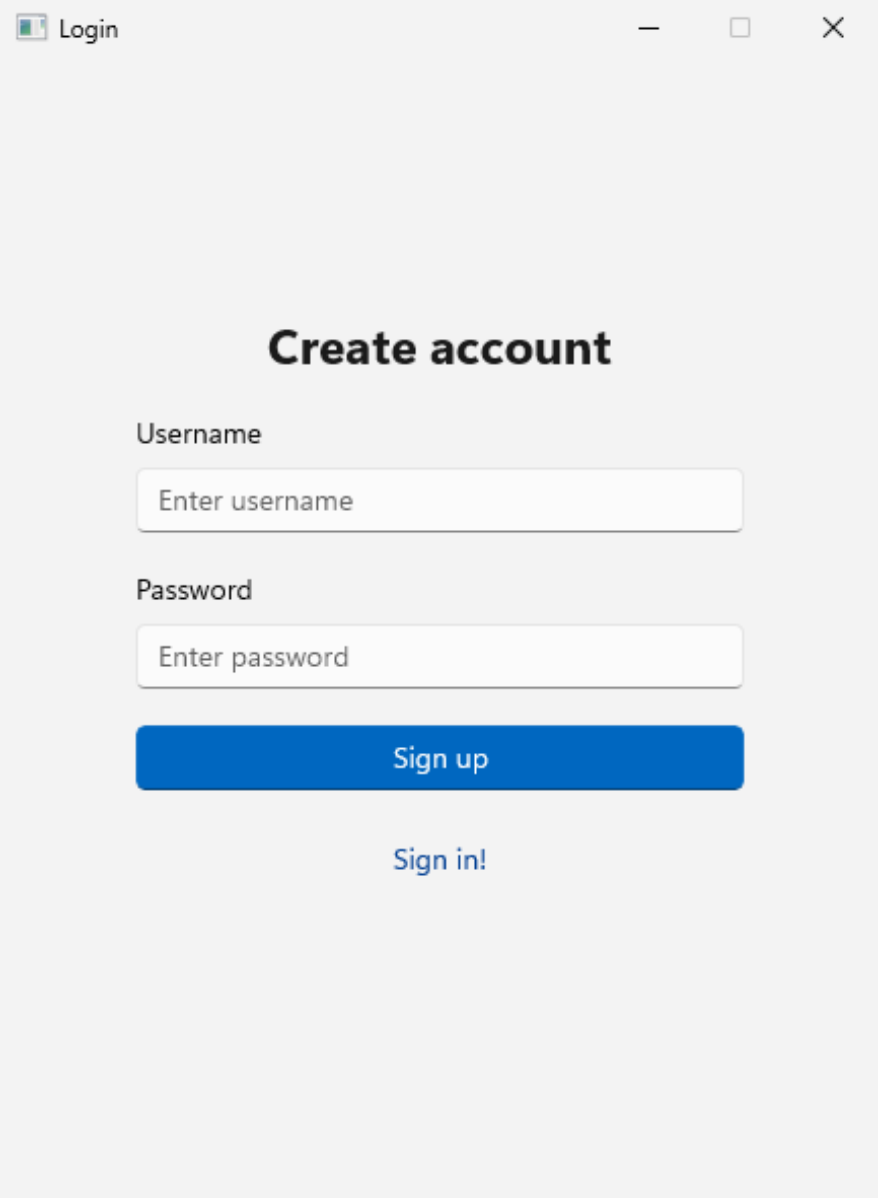
5.1.1. Okno logowania



The image shows a screenshot of a web application's login interface. The window has a title bar with the text "Login" and standard minimize, maximize, and close buttons. The main content area has a light gray background. At the top, the word "Welcome" is displayed in a large, bold, black font. Below this, there are two input fields. The first is labeled "Username" and contains the placeholder text "Enter username". The second is labeled "Password" and contains the placeholder text "Enter password". Below these fields is a prominent blue button with the text "Sign in" in white. At the bottom of the form, there is a link that says "Create account!" in a smaller, blue font.

Rysunek 5.1. Okno logowania

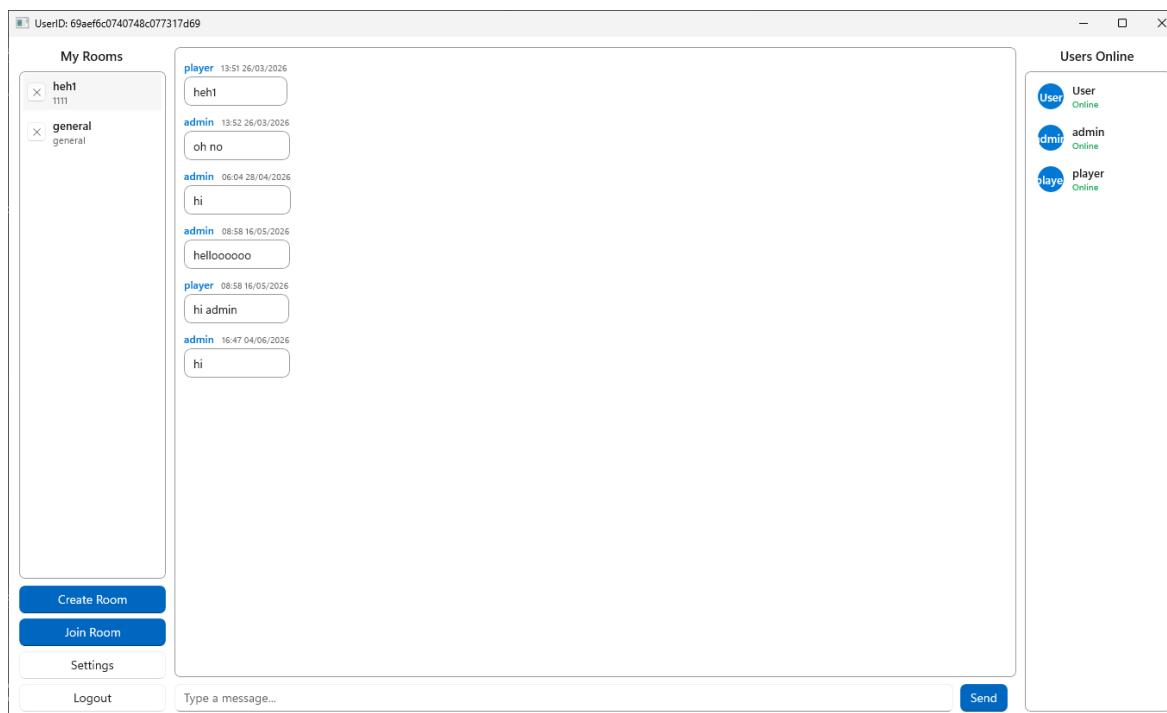
5.1.2. Okno rejestracji



The image shows a screenshot of a web application window titled "Login". The window has a light gray background and standard window controls (minimize, maximize, close) in the top right corner. The main heading is "Create account" in bold black text. Below the heading, there are two input fields: "Username" and "Password". The "Username" field has a placeholder text "Enter username" and the "Password" field has a placeholder text "Enter password". Below these fields is a blue button with the text "Sign up". At the bottom of the form, there is a link that says "Sign in!" in blue text.

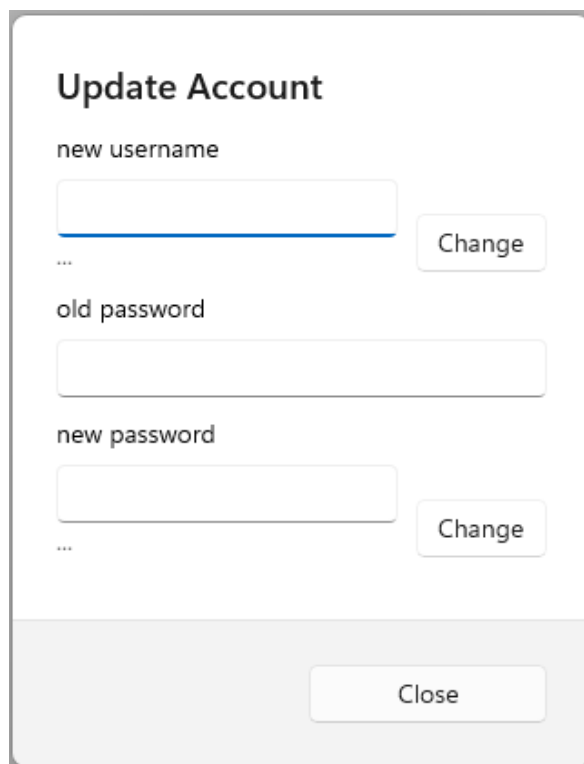
Rysunek 5.2. Okno rejestracji użytkownika

5.1.3. Okno komunikacji tekstowej



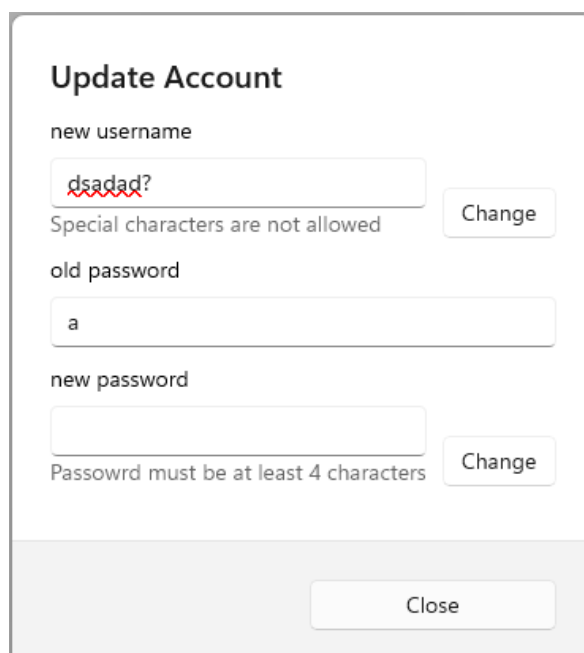
Rysunek 5.3. Główne okno aplikacji

5.1.4. Okno ustawień



The screenshot shows a dialog box titled "Update Account". It contains three input fields: "new username", "old password", and "new password". The "new username" field has a blue underline. To the right of the "new username" and "new password" fields are "Change" buttons. At the bottom right of the dialog is a "Close" button. There are three asterisks (***) below the "new username" field and three asterisks (***) below the "new password" field.

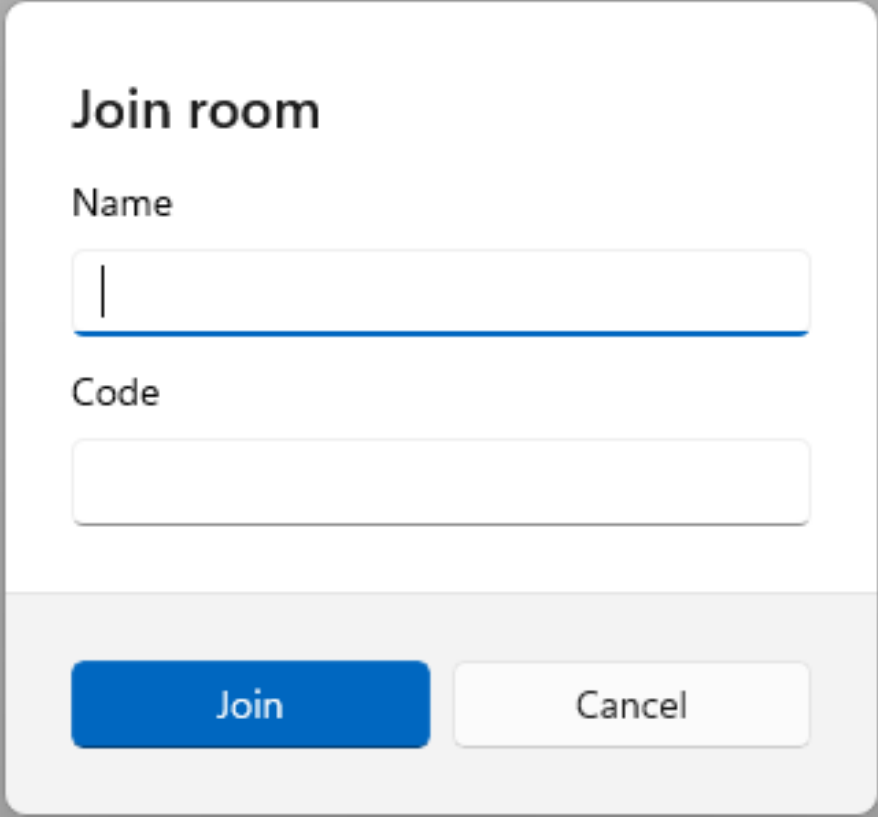
Rysunek 5.4. Okno ustawień, zmiana danych



The screenshot shows the same "Update Account" dialog box, but with validation errors. The "new username" field contains the text "dsadad?" and has a red wavy underline. Below it, the text "Special characters are not allowed" is displayed. The "old password" field contains the text "a". The "new password" field is empty. Below it, the text "Passowrd must be at least 4 characters" is displayed. The "Change" buttons are still present next to the "new username" and "new password" fields. The "Close" button is at the bottom right.

Rysunek 5.5. Błąd zmiany danych

5.1.5. Okno dołączenia do pokoju



The image shows a 'Join room' dialog box. It has a title 'Join room' at the top. Below the title are two input fields: 'Name' and 'Code'. The 'Name' field has a vertical cursor. At the bottom, there are two buttons: 'Join' (blue) and 'Cancel' (white).

Join room

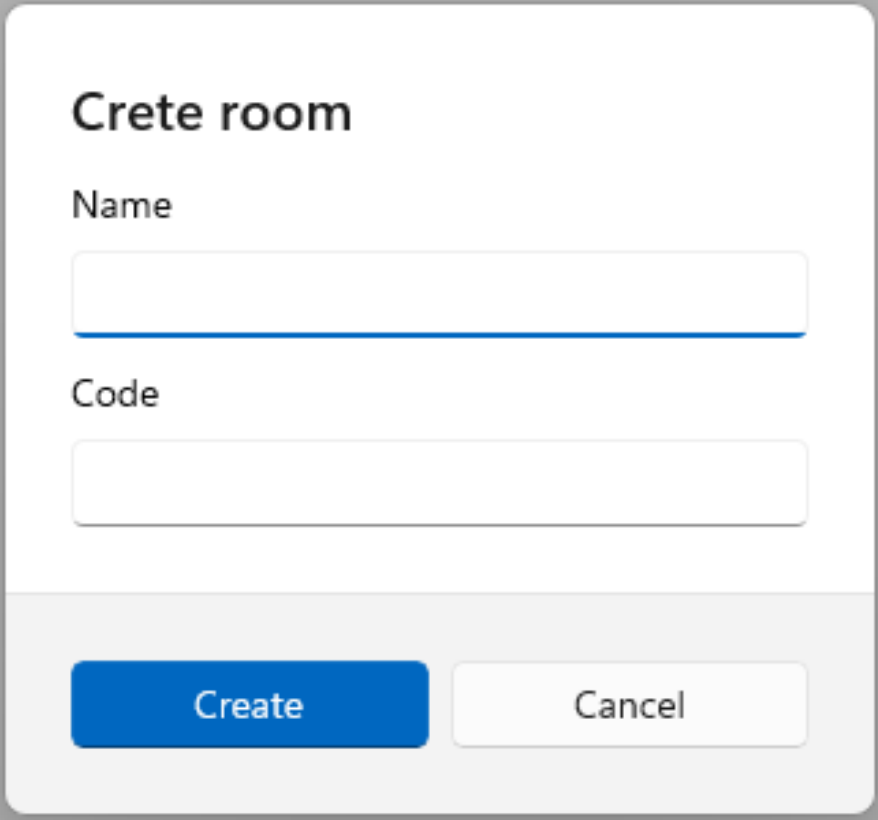
Name

Code

Join Cancel

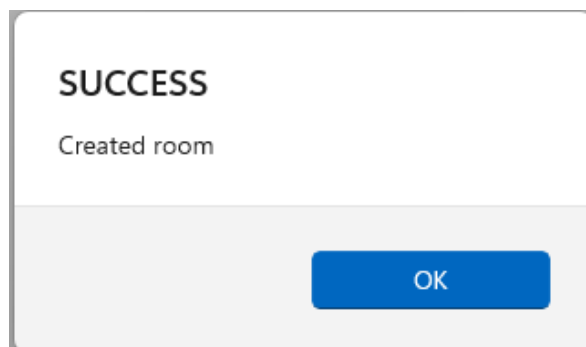
Rysunek 5.6. Okno dołączenia do pokoju

5.1.6. Okno stworzenia pokoju

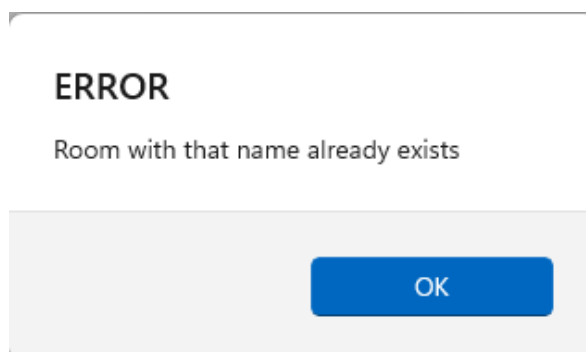


The image shows a 'Create room' dialog box. It has a title 'Create room' at the top. Below the title are two input fields: 'Name' and 'Code'. The 'Name' field is currently selected, indicated by a blue underline. At the bottom of the dialog box, there are two buttons: 'Create' (a blue button) and 'Cancel' (a white button with a gray border).

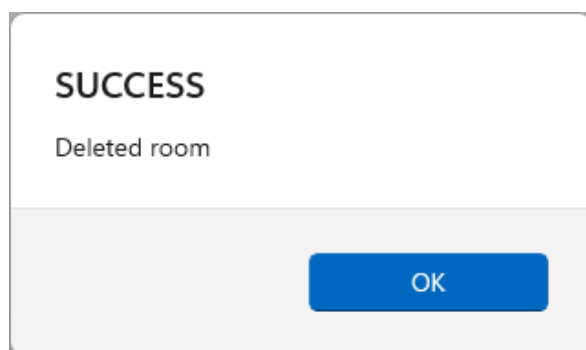
Rysunek 5.7. Okno utworzenia pokoju



Rysunek 5.8. Komunikat udanego stworzenia pokoju



Rysunek 5.9. Błąd tworzenia pokoju



Rysunek 5.10. Komunikat o udanym usunięciu pokoju

5.2. Dane użytkownika dostępu do aplikacji/bazy danych

5.2.1. Baza danych - mongodb.com

- Username: **chat_db_user**
- Hasło: **VbAcD6aY3FPrywXd**

5.2.2. Baza danych - maszyna wirtualna

System:

- Username: **mongo**
- Hasło: **1111**

Database:

- Username: **chat_db_user**
- Hasło: **1111**

5.2.3. Aplikacja

- Username: **admin**
- Hasło: **admin**

5.3. Instrukcja uruchomienia aplikacji

Aby poprawnie uruchomić aplikację do komunikacji tekstowej, należy wykonać następujące kroki:

1. **Bazy danych:** - baza danych jest dostępna w formie zdalnej uruchomiona przez stronę <https://mongodb.com>, lub w postaci maszyny wirtualnej.
 2. **Uruchomienie aplikacji:**
 - (a) W przypadku używania bazy danych w formie eksportu maszyny wirtualnej, należy ją uruchomić przed uruchomieniem serwera i aplikacji oraz w przypadku użycia maszyny wirtualnej należy zmienić adres połączenia z bazą danych odpowiednio do adresu maszyny wirtualnej.
 - (b) Przed uruchomieniem aplikacji należy uruchomić *serwer* poprzez Visual Studio, który będzie łączył się z bazą danych i działał lokalnie na komputerze na adresie *http://localhost:9000/*.
 - (c) Następnie można uruchomić aplikację poprzez Visual Studio lub gotowy zbudowany plik wykonywalny (zaleca się użycie zbudowanego pliku).
 - (d) Aplikacja powinna być gotowa do działania, można przejść do logowania.
-

5.4. Co zostało zawarte w projekcie?

Na potrzeby projektu zaprojektowano i wdrożono nie relacyjną bazę danych w systemie MongoDB. System przechowuje dane użytkowników, wiadomości oraz pokoje do komunikacji.

Aplikacja wykorzystuje interfejs graficzny zrobiony przy użyciu framework'a WinUI.

Aplikacja używa funkcji asynchronicznych co pozwala na jednoczesne działa funkcjonalności w tle (takich jak nasłuchiwanie przychodzących wiadomości, wysyłania wiadomości) bez przeszkadzania w używaniu interfejsu graficznego.

Funkcje CRUD po stronie serwera umożliwiają pełne zarządzanie cyklem życia danych (Create, Read, Delete).

5.5. Możliwości rozwojowe

1. Wdrożenie enkrypcji danych klient → serwer, serwer → klient,
 2. Optymalizacja bazy danych, przechowywać część danych po stronie użytkownika żeby nie obciążać bazy danych,
 3. Możliwość zmiany wyglądu aplikacji poprzez ustawienia użytkownika
-

Bibliografia

- [1] Bcrypt.net. *<https://github.com/BcryptNet/bcrypt.net>*.
- [2] Microsoft entity framework core. *<https://learn.microsoft.com/en-us/ef/core/>*.
- [3] Microsoft .net. *<https://learn.microsoft.com/en-us/dotnet/core/introduction>*.
- [4] Microsoft winui. *<https://learn.microsoft.com/en-us/windows/apps/winui/winui3/>*.
- [5] Mongodb. *<https://www.mongodb.com/docs>*.

Spis rysunków

3.1	NuPackages	6
3.2	Zdjęcie przedstawiające strukturę bazy danych	7
4.1	Diagram Gantta	54
5.1	Okno logowania	55
5.2	Okno rejestracji użytkownika	56
5.3	Główne okno aplikacji	57
5.4	Okno ustawień, zmiana danych	58
5.5	Błąd zmiany danych	58
5.6	Okno dołączenia do pokoju	59
5.7	Okno utworzenia pokoju	60
5.8	Komunikat udanego stworzenia pokoju	61
5.9	Błąd tworzenia pokoju	61
5.10	Komunikat o udanym usunięciu pokoju	61

Spis tabel

Spis listingów

3.1	Table users	7
3.2	Table rooms	8
3.3	Table messages	8
3.4	Database konstruktor	9
3.5	ChatDbContext	10
3.6	Models	11
3.7	Funkcja SendMessageHistoryAsync	12
3.8	Funkcja GetUsersListAsync	13
3.9	Funkcja GetUserRoomsAsync	14
3.10	Funkcja BroadcastToRoomAsync	15
3.11	Funkcja StoreMessageAsync	17
3.12	Funkcja DeleteRoomAsync	18
3.13	Funkcja JoinRoomAsync	19
3.14	Funkcja CreateRoomAsync	20
3.15	Funkcja RegisterUser	21
3.16	Funkcja Authenticate	22
3.17	Program.cs	23
3.18	ConnectionManager.cs	24
3.19	TcpServer.cs	26
3.20	AuthWindow.cs	29
3.21	LoginPage.cs	30
3.22	RegisterPage.cs	33
3.23	ChatWindow-main.cs	36
3.24	OnMessageReceived	38
3.25	RoomListSelectionChanged	42
3.26	LogoutButtonClick	43
3.27	SendMessage	43
3.28	SendButtonClick	44
3.29	MessageInputKeyDown	44
3.30	DeleteRoom	44
3.31	JoinRoom	45
3.32	CreateRoom	46
3.33	Settings	47
3.34	ShowAlert	49
3.35	Message	50
3.36	ChatRoom	50
3.37	User	50
3.38	Tcp	51

Załącznik nr 2 do Zarządzenia nr 228/2021 Rektora Uniwersytetu Rzeszowskiego z dnia 1 grudnia 2021 roku w sprawie ustalenia procedury antyplagiatowej w Uniwersytecie Rzeszowskim

OŚWIADCZENIE STUDENTA O SAMODZIELNOŚCI PRACY

.....Kacper Zoła.....
Imię (imiona) i nazwisko studenta

Wydział Nauk Ścisłych i Technicznych

.....Informatyka.....
Nazwa kierunku

.....134993.....
Numer albumu

1. Oświadczam, że moja praca projektowa pt.: Aplikacja do komunikacji tekstowej

- 1) została przygotowana przeze mnie samodzielnie*,
- 2) nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (t.j. Dz.U. z 2021 r., poz. 1062) oraz dóbr osobistych chronionych prawem cywilnym,
- 3) nie zawiera danych i informacji, które uzyskałem/am w sposób niedozwolony,
- 4) nie była podstawą otrzymania oceny z innego przedmiotu na uczelni wyższej ani mnie, ani innej osobie.

2. Jednocześnie wyrażam zgodę/~~nie wyrażam zgody~~** na udostępnienie mojej pracy projektowej do celów naukowo-badawczych z poszanowaniem przepisów ustawy o prawie autorskim i prawach pokrewnych.

Rzeszów, 07.06.2026
(miejscowość, data)

Kacper Zoła
(czytelny podpis/y studenta/ów)

* Uwzględniając merytoryczny wkład prowadzącego przedmiot

** – niepotrzebne skreślić